

Papers-HQ + Clo-Author

Pipeline Completo para la Producción
Automatizada de Papers Académicos

- 1 Discovery** — Repos + papers (A), datos del usuario + papers (B), o data-first (C)
- 1.5 Data Loading** — Descarga y profiling de datasets candidatos (Path A)
- 2 Ideation** — Generación y ranking de ideas de investigación
- 2.5 Idea Selection** — El usuario elige qué idea desarrollar
- 3.3 Quick Empirical Test** — Verifica identificación en datos reales
- 3 Validation** — Evaluación rigurosa de la idea (score ≥ 7)
- 3.5 Strategy Review** — Aprobación humana de la estrategia
- 3.7 Referee Preview** — Checklist de requisitos + assumption audit
- 4 Strategy & Code** — Intervención manual (Claude en conversación)
- 4.5 Data Audit** — Sanity checks automáticos sobre datos limpios
- Code** — review-paper-code skill + corrección automática
- 4.7 Review** (max 3 rondas)
- 5 Writing** — Redacción del paper (Claude en conversación)
- Peer** — Revisión por pares simulada (6 agentes en paralelo)
- 6 Review**
- 7 Submission** — Paquete de envío a journal

Documento técnico — Pipeline de investigación automatizada
Análisis exclusivamente en Python • 14 stages totales
Última actualización: Mayo 2026

1. Visión General

Este pipeline combina dos sistemas complementarios para producir papers académicos de alto impacto de forma casi completamente automatizada:

Papers-HQ (Stages 1–3.7)

Se encarga de la **fase exploratoria**: descubrir datos y literatura, generar ideas de investigación, validarlas rigurosamente, y generar un checklist de requisitos pre-código. Está compuesto por tres repositorios independientes:

- `search-repositories/` — Discovery (Path A)
- `analyze-data/` — Discovery (Path B)
- `research-junshi/` — Ideation
- `idea-evaluation-pipeline/` — Validation

Clo-Author (Stages 4–7)

Se encarga de la **fase de ejecución**: formalizar la estrategia, correr el análisis econométrico, escribir el paper, someterlo a peer review simulado, y preparar el paquete de envío. **Stages 4 y 5 usan intervención manual** (Claude en conversación), no agentes automatizados.

Checkpoints e Intervenciones Manuales

El pipeline tiene dos checkpoints humanos en las decisiones más críticas de un paper empírico:

Stage 2.5 — Idea Selection: el usuario elige qué idea investigar del top 3 generado automáticamente.

Stage 3.5 — Strategy Review: el usuario aprueba la estrategia de identificación causal antes de la ejecución.

Además, **Stages 4 y 5 usan intervención manual de Claude** (en conversación con el usuario), no subprocesos automatizados.

Principios de diseño

- Automatización donde conviene:** Stages 1, 2, 3, 3.7, 4.5, 6, 7 son automáticos — buscar repos, generar ideas, validar, auditar datos y revisar no requiere intervención.
- Control humano donde importa:** Stage 2.5 (qué investigar) y Stage 3.5 (cómo identificarlo) — las dos decisiones más importantes de un paper empírico.
- Intervención manual (Claude en conversación):** Stages 4 y 5 — la generación de código y escritura del paper se realizan manualmente con Claude en conversación, no mediante agentes automatizados.
- Quality gates en cada fase:** Umbrales determinísticos para avanzar entre stages.
- Python exclusivo:** Todo el análisis econométrico/estadístico usa Python.
- Sin redundancias:** Stage 3 evalúa el diseño de investigación, Stage 6 evalúa el paper completo.

Flujo completo

```
--topic "Economics" [+ --data] o --path-c → Stage 1 (Discovery: Path A, B, o C) →
Stage 1.5 (Data Loading: descarga y profiling, solo Path A) → datasets perfilados →
Stage 2 (Ideation) → Top 3 ideas → Stage 2.5 (Usuario elige) →
Stage 3.3 (Quick Test en datos reales, pre-flight check) →
Stage 3 (Validation, score ≥ 7) → PROMPT.md →
Stage 3.5 (Usuario aprueba) → Stage 3.7 (Referee Preview + assumption audit) → referee_checklist.md →
Stage 4 (Strategy + Code, intervención manual Claude) → results_summary.md →
Stage 4.5 (Data Audit automático) → data_audit.md →
Stage 4.7 (Code Review automático: review-paper-code + corrección max 3 rondas) →
Stage 5 (Writing, intervención manual Claude) → main.tex → Stage 6 (Peer Review, 6 agentes en paralelo) →
Accepted paper → Stage 7 (Submission, score ≥ 85) → Paper PDF final + feedback.pdf
```

Resumen del pipeline

Stage	Nombre	Sistema	Automático?	Lenguaje
1	Discovery (Path A/B/C)	Papers-HQ	✓ Auto	Python
1.5	Data Loading (Path A)	Papers-HQ	✓ Auto	Python
2	Ideation	Papers-HQ	✓ Auto	—
2.5	Idea Selection	Usuario	🧑 Humano	—
3.3	Quick Empirical Test	Papers-HQ	✓ Auto (pre-flight)	Python
3	Validation	Papers-HQ	✓ Auto (loop)	—
3.5	Strategy Review	Usuario	🧑 Humano	—
3.7	Referee Preview	Papers-HQ	✓ Auto	—
4	Strategy & Code	Clo-Author	Manual intervention (Claude)	Python
Papers-HQ + Clo-Author — Explicación del Pipeline				
4.5	Data Audit	Papers-HQ	✓ Auto	Python
4.7	Code Review	Papers-HQ	✓ Auto (review-paper-code)	—

2. Stage 1: Discovery

Objetivo: Obtener una base de datos y 3 papers académicos relacionados que sirvan como semilla para la generación de ideas. El usuario elige uno de dos caminos:

Path A: Búsqueda de repositorios — `search-repositories/`

Se activa cuando el usuario proporciona solo `--topic`.

#	Paso	Detalle
1	Búsqueda en GitHub	Replication packages de papers publicados con mínimo 50 estrellas o 20 forks.
2	Agrupación temática	Los repositorios se agrupan por área temática amplia y se ranquean por star score.
3	Selección de área	El usuario selecciona un área temática de las disponibles.
4	Búsqueda web de papers	3 papers académicos publicados del área seleccionada (Scholar, SSRN, arXiv).

Output A: Repos de GitHub (URL, paper, DOI, dataset) + 3 seed papers.

Path B: Datos del usuario — `analyze-data/`

Se activa cuando el usuario proporciona `--data <URL|archivo>`.

#	Paso	Detalle
1	Carga e inspección	Variables, tipos, N observaciones, cobertura temporal/geográfica, estructura (cross-section, panel, serie de tiempo).
2	Early warning	Si el dataset parece inadecuado (pocas obs., pocas variables, sin variación), se muestra: "⚠ Este dataset tiene N=47 obs. y 3 variables — ¿estás seguro?". El usuario confirma o cambia de dataset.
3	Perfil de datos	Estadísticas descriptivas, patrones de missingness, variables potenciales de tratamiento/outcome.
4	Búsqueda de papers	3 papers relacionados con: (a) el área temática del <code>--topic</code> , (b) el contexto de los datos, y (c) metodologías aplicables a la estructura de datos detectada.

Output B: Perfil del dataset (variables, estructura, summary stats) + 3 seed papers.

Ejemplo: Si el usuario sube un panel de empleo en México 2010-2020 con `--topic "Labor Economics"`, los 3 papers buscados serán sobre economía laboral en LATAM, con metodologías aplicables a datos de panel (DiD, event studies, etc.).

Path C: Data-First — `--path-c` (RECOMENDADO para scores 90+)

Se activa con `--path-c`. Busca los mejores datasets disponibles mundialmente y sugiere topics basados en lo que la data puede soportar.

#	Paso	Detalle
1	Búsqueda curada	Lista priorizada: RCTs primero , luego natural experiments, luego observacional. 12+ paquetes de replicación de top journals (QJE, REStat, etc.).
2	Búsqueda en APIs	Harvard Dataverse + Zenodo con queries de: "RCT replication", "natural experiment", "staggered rollout", "regression discontinuity".
3	Descarga y perfil	Descarga automática, perfila estructura, evalúa score ceiling . Solo datasets con ceiling ≥ 85 califican. RCTs reciben boost automático a ceiling 90+.
4	Sugerencia de topics	Claude sugiere 2 topics por dataset. Cada sugerencia incluye: identification_level (A/B/C), control_group, y score_potential. Nunca sugiere Level C.
5	Selección del usuario	El usuario elige topic + dataset. El pipeline continúa con Path B (data ya descargada).

Output C: Dataset descargado + topic con identificación Level A/B + perfil completo.

Priorización: Tier 1 (RCT/experiment, ceiling 90+) > Tier 2 (natural experiment/staggered DiD, ceiling 85+) > Tier 3 (observational panel, ceiling 75+). Los datasets se prueban en orden de prioridad — RCTs primero.

Handoff a Stage 1.5 (Path A) o Stage 2 (Paths B y C)

→ 3 seed papers (título, autores, DOI, resumen) + recomendaciones de datasets (Path A) o dataset perfilado (Paths B/C)

Modo: Completamente automático. No requiere intervención del usuario.

2b. Stage 1.5: Data Loading

Objetivo: Descargar y perfilar los datasets candidatos identificados por Stage 1 (Path A). Para Paths B y C, este stage se omite porque la data ya está cargada.

Cuándo se ejecuta

Path	¿Stage 1.5?	Razón
A (<code>--topic</code>)	✓ Sí	Stage 1 solo recomienda datasets — Stage 1.5 los baja y los perfila
B (<code>--data</code>)	SKIP	El usuario ya proveyó el dataset, ya fue perfilado en Stage 1
C (<code>--path-c</code>)	SKIP	Stage 1 ya descargó y perfiló como parte del flujo data-first

Proceso (Path A)

#	Paso	Detalle
1	Descarga segura	Para cada dataset recomendado: download a <code>projects/<name>/data/raw/</code> con sanitización de nombres (previene path traversal y nombres reservados de Windows).
2	Detección de formato	Pandas detecta CSV / XLSX / DTA / Parquet / TSV automáticamente según extensión y contenido.
3	Profiling estructural	Shape (filas × columnas), tipos de columnas, identificadores candidatos, dimensión temporal, resumen de NA por columna.
4	Estadísticas descriptivas	Para columnas numéricas: media, mediana, min, max, std. Para categóricas: número de niveles únicos, frecuencias top.
5	Resumen consolidado	Se persiste <code>downloaded_datasets[]</code> en <code>pipeline_state.json</code> con <code>name</code> , <code>local_path</code> , y <code>profile</code> completo.

Output

Archivos: `data/raw/*` (datasets físicos descargados), `pipeline_state.json` (entrada `stage1_5`)

Contenido del state:

- Lista de datasets con paths locales
- Profile estructural por dataset (shape, dtypes, identifiers)
- Summary stats numéricos y categóricos
- Patrones de missingness por columna

¿Por qué un stage separado? En Path A, Stage 1 retorna recomendaciones de datasets (URLs, DOIs) — pero descargar y perfilar puede tardar varios minutos por dataset. Aislarlo permite reanudar con `--from-stage 1.5` si la descarga falló sin re-ejecutar la búsqueda.

Handoff a Stage 2

→ Datasets descargados y perfilados → Stage 2 los usa para constrain ideación a variables que existen realmente

Modo: Completamente automático.

3. Stage 2: Ideation — research-junshi/

Objetivo: Generar y rankear ideas de investigación novedosas basándose en los seed papers de Stage 1.

Input

3 seed papers de Stage 1 (títulos, DOIs, resúmenes) + configuración de área de investigación y venues objetivo.

Proceso

#	Paso	Detalle
1	Lectura de seed papers	Análisis profundo de los 3 papers semilla identificados en Stage 1.
2	Búsqueda en arXiv	Papers publicados en las últimas 24 horas relacionados con los temas de los seeds.
3	Búsqueda en target venues	Exploración de journals específicos del campo para identificar tendencias actuales.
4	Generación de ideas	Se generan entre 8 y 10 ideas de investigación originales.
5	Scoring compuesto	Cada idea recibe un score: $N \times 0.2 + F \times 0.2 + I \times 0.2 + ID \times 0.4$ (Identification-first)
6	Selección top 3	Las 3 mejores ideas se seleccionan para evaluación en Stage 3.

Output

Archivos: stage2_ideation.md , pipeline_state.json (actualizado)

Contenido:

- 8-10 ideas con pregunta de investigación, metodología propuesta, score compuesto
- Conexiones con la literatura existente
- Top 3 ideas rankeadas con justificación
- Ranked ideas JSON para handoff a Stage 2.5

Handoff a Stage 2.5

→ Top 3 ideas rankeadas con scores, justificaciones y datos requeridos → presentadas al usuario

Modo: Completamente automático. La selección la hace el usuario en Stage 2.5.

3b. Stage 2.5: Idea Selection — Human-in-the-Loop

Objetivo: Permitir que el usuario elija qué idea de investigación desarrollar, basándose en el ranking generado automáticamente por Stage 2.

¿Qué se presenta al usuario?

Las **3 mejores ideas** de Stage 2, cada una con:

Elemento	Detalle
Research question	Formulación precisa de la pregunta de investigación
Metodología propuesta	Estrategia de identificación causal sugerida (DiD, RDD, IV, etc.)
Score compuesto	$N \times 0.2 + F \times 0.2 + I \times 0.2 + ID \times 0.4$ (Identification-first)
Justificación del ranking	Por qué esta idea rankea donde rankea vs. las otras
Datos requeridos	Datasets específicos necesarios y su disponibilidad
Conexión con literatura	Papers más cercanos y cómo se diferencia la idea

Decisión del usuario

Decisión	Acción	Resultado
✓ SELECCIONAR	Elegir una de las 3 ideas	La idea seleccionada se formatea como <code>idea_template.txt</code> y pasa a Stage 3
⊕ COMBINAR	Fusionar elementos de 2+ ideas	Se crea una idea híbrida con elementos de múltiples propuestas → Stage 3
✗ RECHAZAR TODAS	Ninguna idea es satisfactoria	Se regresa a Stage 2 con feedback del usuario para generar nuevas ideas

¿Por qué es crítico este paso? El ranking automático ($N \times 0.4 + F \times 0.3 + I \times 0.3$) no puede capturar el conocimiento de dominio del investigador, su familiaridad con los datos, o su agenda estratégica de publicación. Un investigador puede preferir la idea #3 porque tiene acceso privilegiado a los datos, o la #2 porque se alinea con su programa de investigación.

Output

`selected_idea.md` — La idea seleccionada (o combinada) por el usuario, formateada como `idea_template.txt`: título, pregunta de investigación, estrategia de identificación, fuentes de datos, y los 3 papers más cercanos.

Handoff a Stage 3

→ `selected_idea.md` formateada como `idea_template.txt` para el pipeline de evaluación

4. Stage 3: Validation — idea-evaluation-pipeline/

Input: `selected_idea.md` del Stage 2.5 (idea elegida por el usuario)

Objetivo: Evaluar rigurosamente la idea de investigación mediante un pipeline de 8 pasos iterativo hasta alcanzar un score $\geq 7/10$.

Input

Un archivo `idea_template.txt` con: título, pregunta de investigación, estrategia de identificación, fuentes de datos, y los 3 papers más cercanos (con cita completa).

Proceso — Pipeline de 8 pasos con loop

#	Paso	Descripción	Modelo
1	Literature Check	Verificar novedad contra la literatura existente	Sonnet
2	Idea Refinement	Afilar la pregunta de investigación y el alcance	Sonnet
3	Methodology Review	Evaluar la validez de la estrategia de identificación	Sonnet
4	Data Feasibility	Confirmar disponibilidad y adecuación de los datos	Sonnet
5	Pre-Analysis Plan	Fijar especificaciones antes del análisis	Sonnet
6	Draft Evaluation	Evaluar completitud del diseño de investigación	Opus
7	Design Peer Sim	Revisión simulada del diseño (no del paper completo)	Opus
8	Final Score	Asignación del score final 1-10	Opus

Loop: Si el score < 7 , se regresa al paso 2 con feedback. **Máximo 2 pivots** (`MAX_STAGE3_PIVOTS = 2` en `pipeline/config.py`). Si tras agotar los pivots el score sigue < 7 , el pipeline se detiene con aviso: "*⚠ Idea estancada tras 2 rondas de refinamiento (mejor score: X/10). Regresando a Stage 2.5 para seleccionar otra idea.*" Solo si las 3 ideas fueron agotadas se vuelve a Stage 2.

Nota sobre el paso 7: La simulación de peer review en Stage 3 evalúa únicamente el *diseño de investigación* (pregunta, método, datos, viabilidad). No evalúa un paper completo — eso lo hace Stage 6 con el manuscrito terminado. Esto evita redundancia.

Output

Archivos: `stage3_validation.md` , `evaluation/idea.txt` , `PROMPT.md`
Contenido: Idea aprobada (score ≥ 7), pregunta refinada, metodología refinada, pre-analysis plan (PAP), reportes de evaluación de cada paso.

Handoff a Stage 3.5

→ `PROMPT.md` + reportes de evaluación presentados al usuario para revisión de estrategia

Modo: Completamente automático (el loop interno es automático; la intervención humana ocurre en Stage 3.5).

4b. Stage 3.3: Quick Empirical Test

Objetivo: Verificar que la estrategia de identificación propuesta funciona en los datos reales ANTES de invertir tiempo en generación de código y escritura del paper.

Input

Dataset del usuario + idea seleccionada de Stage 2.5.

Proceso — 5 tests automáticos

#	Test	Qué verifica	Si falla
0	Estimator availability	Testa pyfixest, linearmodels, csdid en los datos reales. Registra cuáles funcionan y cuáles no.	Marca estimador como BROKEN — Stage 4 no lo usará
1	Basic DiD	Regresión rápida: $\text{outcome} \sim \text{treat} + \text{entity_FE} + \text{time_FE}$, clustered SEs	Puede indicar efecto nulo o datos inadecuados
2	Permutation test	200 reshuffles del tratamiento. ¿El efecto real está en la cola?	Si $p > 0.1$, la identificación es frágil
3	Entity trends	¿El efecto sobrevive tendencias lineales por entidad?	Si desaparece, es pre-tendencia, no tratamiento

Decisión del usuario

Resultado	Acción
PASS	Todos los tests pasan → proceder con confianza
WARN	Algunos tests pasan con flags → proceder con precaución (-5 score)
FAIL	Permutation o trends eliminan el efecto → usuario elige: (1) Proceder con ceiling ~70, (2) Elegir otra idea (→ Stage 2.5), (3) Proveer nuevos datos

¿Por qué es crítico? En el proyecto oil-democracy, Stage 3.3 hubiera detectado que el permutation $p = 0.588$ y country trends eliminaban el efecto ANTES de generar 4 scripts y un paper de 31 páginas, ahorrando horas de trabajo.

Output

pipeline_state.json actualizado con:
available_estimators: qué paquetes funcionan en estos datos
estimator_warnings: qué falló y por qué
quick_test_results: ATT, permutation p, trends p

Handoff a Stage 3.5

→ Estimadores verificados + test results → presentados al usuario junto con la estrategia

Modo: Completamente automático (~30 segundos).

5. Stage 3.5: Strategy Review — Human-in-the-Loop (2do checkpoint)

Objetivo: Asegurar que el usuario valide la estrategia de identificación causal antes de comprometer recursos en la ejecución.

¿Qué se presenta al usuario?

- **Pregunta de investigación** — formulación exacta
- **Método causal propuesto** — DiD, RDD, IV, matching, etc.
- **Fuentes de datos** — datasets específicos con detalles de acceso
- **Variables clave** — tratamiento, outcome, controles
- **Supuestos clave** — parallel trends, exclusion restriction, etc.
- **Amenazas a la validez** — confounders, selection bias, measurement error
- **Resumen del Pre-Analysis Plan**

Decisión del usuario

Decisión	Acción	Resultado
✓ APPROVE	Proceder a Stage 3.7	Se genera <code>approved_strategy.md</code> como contrato vinculante
↺ REFORMULATE	Ajustar estrategia según feedback	Se re-presenta la estrategia modificada al usuario
✗ REJECT	Regresar a Stage 2.5	El usuario elige otra idea del top 3. Si las 3 fueron agotadas, se vuelve a Stage 2

¿Por qué es crítico este paso? La estrategia de identificación es la decisión más importante de un paper empírico. Un paper técnicamente perfecto con una estrategia débil será rechazado por los referees. Este checkpoint cuesta poco tiempo y previene el error más costoso del pipeline.

Output

`approved_strategy.md` — La estrategia de identificación validada por el usuario. Este documento se convierte en el contrato vinculante para todas las etapas siguientes.

Handoff a Stage 3.7

→ `approved_strategy.md` — estrategia validada por el usuario, lista para referee preview

5b. Stage 3.7: Referee Preview

Objetivo: Simular 2 referees antes del código para generar un checklist de requisitos que el análisis y el paper deberán cumplir.

Input

`approved_strategy.md` de Stage 3.5 — la estrategia de identificación validada por el usuario.

Proceso — Dos referees simulados + assumption audit

Domain Referee

Evalúa desde la perspectiva del campo:

- Contribución a la literatura
- Gaps en la literatura que se deben cubrir
- Construcción y fuentes de datos
- Tablas y figuras necesarias para convencer

Methods Referee

Evalúa la solidez metodológica:

- Estrategia de inferencia causal
- Tests de especificación requeridos
- Checks de robustez necesarios
- Pitfalls y errores comunes a evitar

Assumption Audit por método (NUEVO): Checklists específicos según el diseño causal:

DiD: parallel trends, no-anticipation, composición estable, HonestDiD sensitivity

IV: first-stage $F > 10$, exclusión restriction, monotonicity, Anderson-Rubin CIs

RDD: McCrary density test, bandwidth sensitivity, donut-hole, continuidad de covariables

RCT: balance table + joint F-test, permutation inference, attrition, Lee bounds, Cohen's d

SC: pre-treatment RMSPE, predictor balance, donor pool justification, permutation inference

Filtro de estimadores: El checklist se cruza contra los estimadores verificados en Stage 3.3. Si un estimador está marcado como BROKEN, los requisitos que lo necesitan se marcan como [DOWNGRADED] y no se prometen en Stage 4.

Output

`strategy/referee_checklist.md` — Checklist estructurado con las siguientes categorías:

- **MUST-HAVE:** Requisitos indispensables que el paper debe cumplir
- **SHOULD-HAVE:** Elementos altamente recomendados
- **NICE-TO-HAVE:** Elementos opcionales que fortalecen el paper
- **WARNINGS:** Riesgos y amenazas identificadas por los referees
- **MUST-NOT-CLAIM:** Claims que el paper no debería hacer dado el diseño

Diferencia con Stage 3 y Stage 6: Stage 3 evalúa si la *idea* es viable; Stage 3.7 genera un *checklist de requisitos* para guiar la implementación; Stage 6 evalúa el *paper completo*. No hay redundancia: cada stage opera sobre un artefacto distinto en un momento distinto del pipeline.

Handoff a Stage 4

→ `referee_checklist.md` — checklist de requisitos que guía la generación de código y escritura en Stages 4-5

Modo: Completamente automático.

6. Stage 4: Strategy & Analysis — Intervención Manual (Claude en conversación)

Objetivo: Formalizar la estrategia aprobada y ejecutar el análisis econométrico completo en Python. **Este stage usa intervención manual:** Claude (en conversación con el usuario) escribe los artefactos y genera los scripts.

Input

- `approved_strategy.md` de Stage 3.5 — la estrategia validada por el usuario
- `strategy/referee_checklist.md` de Stage 3.7 — requisitos de los referees simulados

Sub-fase 4a: Formalización de la estrategia (Claude en conversación)

Importante: Claude NO rediseña la estrategia — solo la formaliza y traduce a artefactos implementables. El usuario supervisa y puede corregir en tiempo real.

Claude produce:

- `strategy_memo.md` — estimand, estimador, supuestos de identificación
- `pseudo_code.md` — paso a paso para la implementación en Python
- `robustness_plan.md` — especificaciones alternativas, placebos, falsificación

Sub-fase 4b: Generación y ejecución de scripts (Claude en conversación)

Claude genera 4 scripts Python y los ejecuta:

Script	Descripción	Packages Python
00_clean.py	Carga, limpieza y construcción de variables	<code>pandas</code> , <code>polars</code>
01_main.py	Especificación principal	<code>pyfixest</code> , <code>linearmodels</code> , <code>statsmodels</code>
02_robustness.py	Checks de robustez	<code>pyfixest</code> , <code>doubleml</code> , <code>rdrobust</code>
03_output.py	Tablas y figuras publication-ready	<code>stargazer</code> , <code>matplotlib</code> , <code>seaborn</code>

Claude ejecuta los scripts y señala finalización escribiendo `intervention_done.json` .

Validación automática del pipeline

El pipeline valida automáticamente que:

- Los 4 scripts existen y corrieron sin errores (exit code 0)
- `clean_data.csv` fue producido correctamente
- Tablas LaTeX generadas en `paper/tables/*.tex`
- Figuras generadas en `paper/figures/*.pdf`
- `results_summary.md` contiene point estimates, SEs, sample sizes

Output obligatorio

- `scripts/python/` — Scripts numerados (00, 01, 02, 03)
- `paper/tables/*.tex` — Tablas LaTeX publication-ready
- `paper/figures/*.pdf` — Figuras en PDF y PNG
- `results_summary.md` — **Artefacto obligatorio de handoff:** point estimates con SEs, sample sizes, estadísticas clave, resultados de robustez
- `intervention_done.json` — Señal de finalización

6b. Stage 4.5: Data Audit

Objetivo: Sanity checks automáticos sobre `clean_data` ANTES de escribir el paper. Detectar problemas en los datos que podrían invalidar los resultados.

Input

`clean_data.csv` producido por Stage 4 + `results_summary.md`.

Proceso — 4 checks automáticos

#	Check	Detalle	Criterio de flag
1	Distribution checks	Distribución de variables clave vs. rangos esperados	Variables fuera de rangos plausibles → flag de posible error de codificación
2	Cell sizes	Tamaños de celda por grupo de interacción (tratamiento × subgrupo)	N < 30 en alguna celda → flag de poder estadístico insuficiente
3	Outcome distribution	Distribución del outcome: zero rate, skewness, outliers extremos	Zero rate alto, skewness extrema, o outliers > 3 SD de la media
4	Outlier detection	Observaciones con valores extremos que distorsionan la estimación	Valores > 3 SD, ratio max/min extremo, o distribución muy sesgada

Output

`data_audit.md` — Reporte completo de los 4 checks con:

- Resultado de cada check (PASS / WARNING / CRITICAL)
- Estadísticas descriptivas de soporte
- Recomendaciones de acción para cada flag

Flags CRITICAL: Si algún check produce un flag CRITICAL, el pipeline **advierte al usuario antes de proceder a Stage 5**. El usuario decide si continuar, ajustar los datos, o regresar a Stage 4 para re-limpiar.

Uso posterior: `data_audit.md` se incluye en el evidence packet que reciben los referees en Stage 6, proporcionando evidencia de que los datos fueron auditados antes de la escritura del paper.

Handoff a Stage 4.7

→ `data_audit.md` — reporte de auditoría de datos, listo para ser referenciado en el paper y consumido por el code review

Modo: Completamente automático.

6c. Stage 4.7: Code Review — review-paper-code skill

Objetivo: Revisar automáticamente los scripts generados en Stage 4 antes de pasar a la escritura del paper, detectando problemas de reproducibilidad, calidad de código, y desalineación entre el código y lo que prometen los referees del Stage 3.7. Si encuentra problemas críticos, los corrige y vuelve a revisar.

Input

- scripts/python/*.py — Scripts numerados generados por Stage 4 (1_load, 2_clean, 3_analyze, 4_output)
- strategy_memo.md — Estrategia formalizada (para verificar alineación)
- strategy/referee_checklist.md — Requisitos MUST/SHOULD/NICE de Stage 3.7
- data/clean/*.csv + main_results.csv — Outputs producidos por los scripts

Proceso — 2 agentes revisores en paralelo

Reproducibility & Quality

- ¿Los scripts corren end-to-end sin error?
- ¿Hay seeds fijados donde corresponde?
- ¿Los paths son relativos / portables?
- Estructura, claridad, comentarios, código comentado huérfano
- Transformaciones opacas o thresholds mágicos sin justificar

Paper-Code Alignment

- Tablas .tex vs. lo que producen los scripts
- Variables del paper vs. variables del código
- Sample restrictions, métodos, clustering, fixed effects
- Cumplimiento del referee_checklist
- Estimadores marcados como BROKEN en Stage 3.3 no usados

Loop de auto-corrección

Fase	Acción
1. Backup	Antes de cada ronda, los scripts se copian a scripts/python/backup_r1/ , backup_r2/ , etc. Permite rollback si una corrección rompe más de lo que arregla.
2. Detección	Si la review detecta issues críticos (reproducibilidad rota, claim del paper sin código que lo respalde, etc.) entra al loop de corrección.
3. Corrección	Un agente correctivo modifica los scripts según los issues detectados.
4. Re-review	Vuelve a correr la revisión completa con los scripts corregidos.
5. Cap	MAX_CODE_REVIEW_ROUNDS = 3 en pipeline/config.py . Si tras 3 rondas siguen quedando issues críticos, se reportan al usuario para intervención manual.

Skill base: Skills/review-paper-code.md (originalmente desarrollado por [Claes Bäckman](#)). El pipeline lo invoca via claude -p con dos prompts especializados (reproducibilidad / alineación) corriendo en paralelo.

Output

quality_reports/code_review_report.md — Hallazgos consolidados con prioridad
scripts/python/backup_r{N}/ — Snapshots por ronda de corrección
scripts/python/*.py — Versión corregida (in-place, con backups disponibles)
Estado en pipeline_state.json : stage4_7 con issues, fixes aplicados, score final

Handoff a Stage 5

→ Scripts validados → Stage 5 puede confiar que los outputs (tablas, figuras, results_summary) están alineados con el paper que va a escribir

Modo: Completamente automático.

7. Stage 5: Writing — Intervención Manual (Claude en conversación)

Objetivo: Redactar un manuscrito LaTeX completo a partir de los resultados del análisis. **Este stage usa intervención manual:** Claude (en conversación con el usuario) escribe todas las secciones del paper.

Input

- `results_summary.md` — Resultados principales, SEs, effect sizes
- `paper/tables/*.tex` + `paper/figures/*.pdf` — Tablas y figuras de Stage 4
- `strategy_memo.md` — Estrategia de identificación formalizada
- `data_audit.md` — Reporte de auditoría de datos de Stage 4.5
- `strategy/referee_checklist.md` — Requisitos de los referees de Stage 3.7

Proceso — Claude en conversación escribe todas las secciones

#	Sección	Guía
1	Introduction	Contribución al campo, preview de resultados, roadmap del paper
2	Literature Review	Posicionamiento frente a la frontera de investigación
3	Data	Fuentes, muestra, estadísticas descriptivas (referencia a tablas)
4	Empirical Strategy	Identificación, supuestos, amenazas a la validez
5	Results	Resultados principales + robustez, effect sizes con unidades
6	Conclusion	Implicaciones, limitaciones, futuras líneas de investigación
7	Abstract	Máximo 150 palabras — se escribe al final
8	Humanizer pass	Eliminación de patrones de escritura de IA (hedging, filler, repetición)

Compilación automática

Pipeline de compilación: `pdflatex` × 3 pasadas + `bibtex` → PDF. Si la compilación LaTeX falla, se usa fallback con `fpdft2` para generar un PDF funcional.

Estándares de formato

Documento

- LaTeX, `\documentclass[12pt]{article}`
- Márgenes: 1 pulgada (`geometry`)
- Cuerpo: `\doublespacing`
- Bibliografía: `biblatex` + `biber`

Tablas y figuras

- Tablas: `booktabs` (toprule, midrule, bottomrule)
- Notas obligatorias en toda tabla y figura
- Captions: font-small, bold labels
- `hyperref` cargado último

Modo R&R

Si Stage 6 solicita revisiones, Claude reescribe las secciones afectadas basándose en el feedback de los referees.

Output

`paper/main.tex` — Manuscrito principal (source of truth)
`paper/main.tex` — Archivo único con todas las secciones (ya no usa sections/)
`Bibliography_base.bib` — Bibliografía centralizada

Handoff a Stage 6

→ `paper/main.tex` compilado a PDF para el proceso de peer review

8. Stage 6: Peer Review — 6-agent review-paper skill

Objetivo: Simular una revisión por pares rigurosa lanzando **6 agentes especializados en paralelo** que cubren distintos focos del paper, y aplicando **umbrales determinísticos** sobre el promedio de scores para decidir el destino editorial.

Input

`paper/main.tex` compilado como PDF, junto con los artefactos de Stage 4 / 4.5 / 4.7 que sirven como evidence packet.

Sub-fase 6a: Pre-review consistency check

Antes de lanzar los 6 agentes, el pipeline lee todas las secciones, tablas y CSVs de resultados, y auto-corrige contradicciones evidentes (números que no matchean entre tabla y texto, citations rotas, símbolos inconsistentes). Esto evita que los agentes pierdan tiempo reportando problemas triviales.

Sub-fase 6b: 6 agentes especializados (en paralelo)

#	Agente	Foco
1	Spelling & Grammar	Ortografía, gramática, estilo académico, registro formal
2	Internal Consistency	Cross-references, números entre tabla / texto / abstract, símbolos consistentes
3	Unsupported Claims	Afirmaciones causales sin respaldo, integridad de la identificación, overclaiming
4	Mathematics & Notation	Ecuaciones, notación, derivaciones, definiciones de variables
5	Tables & Figures	Documentación, footnotes, unidades, referencia desde el texto
6	Contribution Referee (adversarial)	Evaluación adversarial de contribución, fit al journal, novedad

Evidence packet (todos los agentes reciben): `data_audit.md`, `results_summary.md`, `cell_sizes.csv`, tablas `.tex` reales del código, code review report de Stage 4.7. Esto garantiza que los agentes verifiquen claims contra la evidencia, no contra prácticas asumidas.

Implementación: los 6 prompts corren en paralelo via `concurrent.futures` con `MAX_PARALLEL_AGENTS` (default 2). El agente 6 (contribution referee) tiene timeout extendido por la complejidad de la evaluación adversarial.

Sub-fase 6c: Decisión editorial — Umbrales determinísticos

El editor clasifica cada concern (FATAL / ADDRESSABLE / TASTE) y aplica umbrales determinísticos:

Condición	Decisión
avg_score $\geq$ 75 AND no fatal	ACCEPT
avg_score $\geq$ 75 AND fatal	MINOR REVISIONS
avg_score 60-74 AND no fatal	MINOR REVISIONS
avg_score 60-74 AND fatal	MAJOR REVISIONS
avg_score 40-59	MAJOR REVISIONS
avg_score < 40 AND fatal	REJECT

R&R loop: Máximo 3 rondas. En ronda 2+, referees verifican si concerns anteriores fueron addressed. Tras agotar las rondas de R&R, el usuario siempre recibe 3 opciones:

- [1] Aceptar score actual → Stage 7
- [2] Manual fix → re-review
- [3] Restart desde Stage 3.7 (reconstruir código + paper)

Output

`quality_reports/reviews/` — 6 reportes individuales + síntesis editorial
`response_letter.md` — Carta de respuesta punto por punto (si hay R&R)
Estado: `stage6` con scores por agente, decisión, y número de ronda R&R

9. Stage 7: Submission — Clo-Author

Objetivo: Preparar el paquete final de envío a journal.

Input

Paper aceptado (post-peer review) + todos los artefactos del pipeline.

Proceso

Agentes: orchestrator + verifier

#	Paso	Detalle
1	Journal targeting	Lookup table instantáneo basado en score (no usa Claude). Recomienda journal primario + backup.
2	Paquete de replicación	Auditoría AEA Data Editor compliant: todos los scripts Python corren sin error, datos documentados con codebook, README.
3	Verificación de scripts	Todos los scripts en <code>scripts/python/</code> se ejecutan end-to-end sin errores.
4	Documentación	Codebook completo, fuentes documentadas, requisitos computacionales.
5	Final quality gate	Score agregado ≥ 85 , todos los componentes ≥ 70 . Scores de code/paper recalculados dinámicamente.
6	Feedback PDF	Diagnóstico visual: radar chart, robustness table, referee issues, recomendaciones priorizadas.
6	Compilación final	PDF final compilado con pdflatex (3 pasadas + bibtex).

Decisión del usuario: Si el score no alcanza el gate, el usuario puede: **Accept** (aceptar el score actual) o **Improve** (máximo 3 rondas adicionales de mejora, con depth limit).

Output

`paper/main.pdf` — Paper final submission-ready
`paper/replication/` — Paquete de replicación AEA-compliant
`journal_targeting.md` — Recomendación de journal con justificación
`feedback.pdf` — Reporte visual con radar chart, robustness table, y recomendaciones

10. Sistema de Calidad

Los pesos están ajustados para evitar redundancia con las evaluaciones de Papers-HQ (Stages 1-3.7), que ya cubren literatura y datos:

Componente	Peso	Fuente
Validez de la identificación	30%	strategy_memo.md + referee_checklist.md
Calidad del paper	25%	Promedio domain-referee + methods-referee
Calidad del código	20%	Validación automática (scripts OK, outputs generados)
Polish del manuscrito	15%	Peer review Stage 6
Readiness de replicación	10%	verifier (pass/fail $\rightarrow$ 0 o 100)

Thresholds

Gate	Score	Requisito
CRITIC_GATE	≥ 70	—
ACCEPT_GATE	≥ 75	No fatal
MINOR_REV_GATE	≥ 60	—
REJECT_FLOOR	< 40	+ fatal
SUBMISSION_GATE	≥ 85	Todos ≥ 70
COMPONENT_MIN	≥ 70	—

Escalación

Dado que Stages 4 y 5 usan intervención manual (Claude en conversación), la escalación es directa al usuario:

Situación	Acción
Scripts fallan validación	Claude corrige en conversación
Data audit con flags CRITICAL	Usuario decide si continuar
Peer review con REJECT	Usuario elige: fix, restart, o aceptar

14 stages, 2 checkpoints humanos (qué investigar + cómo identificarlo), 2 stages de intervención manual (código + escritura), el resto automático. Incluye: Stage 1.5 (Data Loading para Path A), Stage 3.3 (Quick Empirical Test pre-flight), Stage 4.7 (Code Review automático con review-paper-code skill), identification-first scoring (ID $\times$ 0.4), assumption audit por método, outcome validation, falsification planning, sanity checks, writing standards, peer review por 6 agentes en paralelo, feedback.pdf visual, y scores dinámicos. Gate: aggregate ≥ 85 , cada componente ≥ 70 . Todo el análisis econométrico en Python.